

AI LIVE TRANSLATOR

Carlos Lomelín Valdés

Sebastián Garmendia Rivera

University of Coimbra

uc2025154763@student.uc.pt, uc2025155216@student.uc.pt

1 Introduction

The AI Live Translator is a real-time, multi-agent translation system, designed to contribute and enable fluent communication between speakers of different languages.

1.1 Problem

Communication is the cornerstone of development between individuals, communities and society itself. This practice allows for the constant dissemination of feelings, information and ideas.

Across evolutionary time, it can be appreciated how living organisms have progressively developed more abstract and efficient communicative systems, leading to the flourishing of their species.

That growth leaves the human species in reality with more than 7,000 languages spoken around the globe, representing an enormous cultural wealth, as well as a challenge when trying to communicate with someone whose language is different from yours.

This dilemma and barrier arise in diverse contexts such as professional collaboration, international travel, academic settings, everyday life, among others. Underscoring the need for effective mechanisms to bridge language differences.

1.2 Motivation

Both authors of this document find ourselves, to varying degrees, confronted by the aforementioned problem. Our native language is Spanish, and we are currently participating in a student exchange in Portugal, where the preferred language of instruction and daily interaction is Portuguese.

While we possess sufficient proficiency to understand and sustain basic conversations, there are recurrent moments where we are unable to efficiently follow or contribute to the actual discussion. This occurs particularly in fast-paced or technical contexts.

This challenge is not unique to us. Thousands of students, professionals, and travelers worldwide face similar or more acute barriers. Therefore, we want to create a tool that allows anyone in the world to communicate fluently in real time, without the need of requiring prior study or advanced linguistic preparation.

Taking into consideration the outlined above, our settled goal is to propitiate inclusive, practical and efficient development in academic and professional environments involving any existing language.

1.3 Summary of Multi-Agent and selected course areas

The multi-agent system described in this document is capable of real-time conversation translation through the synchronization of seven agents powered by nine local developed tools. Together, they manage the full pipeline, from audio capture and segmentation to recognition, translation, and speech synthesis, within a streamlined user interface in a web application based on Streamlit, a free and open-source Python library. In general terms, the interface allows the user to save a profile and, according to their preferences, perform audio and text translation of the conversation they are currently experiencing.

It is worth noting that four third-party tools were used to perform specific tasks such as audio-to-text conversion, translation, and text-to-audio transformation. These external components are invoked selectively to balance quality and responsiveness, while the agent layer coordinates all the tasks throughout the system. In practice, this design enables fluid, back-and-forth dialogue across languages, making the system suitable for classrooms, daily interactions and professional (but no technical) contexts.

Regarding the course topics selected for this project, constraint satisfaction problems (CSP) and search methods, both were incorporated to enhance the system's performance. CSPs were implemented outside the core agent flow to enforce restrictions between specific language pairs, where direct translations could reduce accuracy, as well as between certain voice profiles and target languages, since the system allows generating final audio outputs with different voices. When a restricted configuration is detected, the CSP module triggers a message before execution to inform the user of potential accuracy issues.

In contrast, the depth-first search (DFS) method, which is a search-based technique, is fully integrated within the agent flow and serves as a central mechanism to improve translation accuracy. DFS is applied to avoid errors in restricted lan-

guage pairs by routing translations through intermediate languages that provide more faithful results. For example, if translating directly from X to Z is likely to reduce accuracy, DFS dynamically determines an alternative path such as $X \rightarrow Y \rightarrow Z$, where Y is an intermediate language that preserves the semantic fidelity of the source content. By embedding DFS within the agent workflow, the system ensures that translations are both accurate and coherent while maintaining seamless interaction between agents.

1.4 Initial Analysis of Results

The initial experiments provide a clear overview of the system's performance across different translation scenarios. The architecture consistently handled cases involving restricted language paths, where the DFS agent successfully computed alternative routes (e.g., $es \rightarrow en \rightarrow ja$) and enabled accurate final translations. Interface-level constraints also behaved as intended, preventing redundant operations and alerting users to potential accuracy issues, thereby improving interaction reliability.

The system demonstrated robust handling of long inputs: when translations became extensive, the summarizer agent generated concise outputs that preserved the main semantic content, offering an efficient alternative for tasks requiring rapid comprehension. However, the experiments also revealed limitations related to the use of a free model, including rate-limit interruptions and occasional inconsistencies in JSON formatting or instruction adherence.

Qualitative evaluation showed strong fidelity, naturalness, and contextual coherence across languages. Quantitatively, back-translation analysis yielded low error rates, with most discrepancies concentrated in a small number of upstream deviations. Overall, the preliminary results confirm that the system effectively fulfills its primary objective of producing reliable multilingual audio translations while highlighting areas for improvement in model stability and resource availability.

2 Multi-Agent System Architecture

To carry out this project, we use the Microsoft AutoGen Studio framework, which allows us to create and shape our different agents that are necessary for performing multiple tasks, forming a multi-agent system. This framework was selected because it is the one we are most familiar with and has an intuitive workspace, thus allowing for the proper development of the AI Live Translator.

According to the multi-agent architecture, below we present a list of the agents involved, their roles and the tools they implement.

• Memory/Profile Agent

This agent loads and updates the user's profile, delivering the information to the rest of the pipeline. Including the language spoken, the language to be translated, the formal/informal context, the gender of the audio translation and the maximum latency.

Tools: *user_profile* (Data Bases in SQL Lite) - A database was developed to allow the creation and updating of user profiles, containing the preference data of each user of the model

so that its use is appropriate to the stated priorities. This was done directly in Python, acting as a "backend".

• ASR Agent

The Automatic Speech Recognition agent is responsible for receiving the audio input and transcribe it into text.

Tools: *auto_text_recognition* (implement Whisper).

Third Party Tools: Whisper from OpenAI - this tool is an open-source automatic speech recognition system that accurately transforms audio into text for each input entered into the UI by recognizing pauses, noises, filler words, and multiple languages. Specifically, it was implemented locally, running on its base model, to ensure an acceptable response speed.

• Latency Management Agent

The agent with the latency task is in charge for estimating the translation time according to the length of the transcribed text, thus deciding which translation engine or technique the MT Agent uses. This ensures that the TTS Agent delivers an output of acceptable quality (translating literally), but prioritizes time, or excellent translation quality (seeking to identify the context in which the message and the conversation is taking place) if time is not critical. All this, in accordance with a pre-established maximum time per translation (SLA – Service Level Agreement).

Tools: *latency* - this tool works by defining two specific values (a default system latency of 1500ms and a text length of 80 characters). If the user-selected latency is less than or equal to the system default, and the text to be translated is less than 80 characters, an index indicating "fast mode" along with the latency time is provided. Otherwise, an indicator requesting "quality mode" is given. These returns are managed by the Latency Agent and sent to the MT Agent to decide whether the translation should prioritize speed or quality.

• DFS Agent

The Depth First Search (DFS) Agent receives the transcription produced by the ASR Agent and invokes the *dfs_translate* function when necessary. This function determines whether a transitional translation is required to improve accuracy between non-similar language pairs. Using a graph representation of languages, the DFS method identifies an optimal path through intermediate languages, ensuring a more faithful translation. Once the path is determined, the *translate_along_path* function executes the translation step by step along the transitional languages, leveraging a third-party translation model (Google Translator), which is the same model used by the MT Agent and is described in detail in that section. Finally, the DFS Agent returns the translated transcript along with the source language corresponding to the final intermediate node in the path.

Tools: *dfs_translate* - this function forms the core of the DFS Agent. It analyzes whether transitional translations are necessary, applies the DFS algorithm to identify an accurate path through the language graph, and coordinates the translation along that path via *translate_along_path*. By systematically using intermediate languages, it maximizes translation fidelity between language pairs that may otherwise produce errors if translated directly.

• MT Agent

The Machine Translation Agent is in control of translating the text received by the DFS Agent, according to the remaining latency and the preferences of the profile in use.

Tools: *machine_translation* (implements Google Translator).

Third Party Tools: Google Translator and Alibaba Tongyi DeepResearch 30B A3B – Google Translator is the most widely used free translation engine in the world, providing reliability and access to a vast repertoire of languages. This allows for the generation of a translation acceptable to the user in a local manner (the tool was always implemented locally on the tests machines). Likewise, Alibaba Tongyi DeepResearch 30B A3B is employed, which, thanks to its trained artificial intelligence model, allows for more detailed translation. This tool is operated if the client wants to generate a more detailed translation (in situations where there is a higher latency margin and with a larger amount of content to translate). In contrast, this last tool was implemented from an API key, so its use was remote as it depended on the servers of the company in question.

• Summarizer Agent

Once the translation by the MT Agent is ready, the Summarizer Agent makes a summary in text about what was translated to give the user the option of having a shorter and more direct version of what the other person is trying to tell them. The instruction given to the Third Party Tool was to make a summary only in case that the text size is bigger than 150 characters.

Third Party Tool: Alibaba Tongyi DeepResearch 30B A3B (via provider or Open Router) with prompts of compression.

• TTS Agent

Finally, the Text to Speech Agent is responsible for converting the text to audio from the original translation performed by the MT Agent. Initially, the output is only the original text and audio translation, with the user being given the option to see the summarized text in case the text has a large length.

Tools: *text_to_speech* (implement ElevenLabs).

Third Party Tools: ElevenLabs - this service was not run locally, but rather through an API key. Generally speaking, ElevenLabs is an artificial intelligence developer that focuses on audio processing to mimic voices, generate sound effects, and other applications. In this particular case, it helped convert the translated text into audio with certain predefined characteristics such as audio speed, pitch, and voice gender.

These different agents are coordinated through the use of agentic patterns that are implemented across the developed code. Regarding the defined operation, we employ a sequential pattern due to the linear and sequential workflow of the agents when transcribing the input, translating it (depending on the context, formality, constraints and delay time that will occur), writing or summarizing the output to subsequently generate an audio of it. In second instance, as could be seen in the list above, a tool use pattern is implemented as our project need to delegate specialized tasks rather than performing everything “internally”. In addition, a multi-agent pattern is demanded since it is the project basis, and the workflow of

this system requires breaking down the main objective into sub duties where each of the agents presented come into action. Indeed, a human in the loop pattern is also key for the AI Translator due to the wait that the model has to carry out in every input of the conversation (every time a human speaks and listens).

It should be emphasized that the architecture mentioned above is mostly based on third party models, as this allows us to use tools and mechanisms with more developed knowledge. Moreover, these models contribute to obtain better results in terms of quality and speed, due to their capacity to use cutting-edge technology, manage low latency rates and work with continual updating and optimization.

3 Materials and Technical Details

For the development of AI Live Translator, we first defined the language in which we work, which is Python, since it has the versatility to work with artificial intelligence thanks to its wide selection of libraries and frameworks, thus providing functionalities that enhanced our project.

3.1 Dependency Block

Subsequently, we began by defining our dependency block, which first included importing *asyncio*, *os*, *re*, *json*, *ast* and *random*. These are used to run asynchronous functions (multi-agent group), read environment variables, search text blocks, parse in JSON format, give a safe way to convert text literals to Python objects and in the selection of random solutions, respectively. Next, we called *AssistantAgent* and *RoundRobinGroupChat* to orchestrate the agents so they communicate or execute in turns, and *TextMentionTermination* to give the agents a stop condition when "TRANSLATION" is displayed. We also imported *Console*, which was useful for visualizing the chat before implementing Streamlit, and finally *OpenAIChatCompletionClient*, which allowed us to connect to the OpenAI model “alibaba/tongyi-deepresearch-30b-a3b:free” via the previously generated API key. All of this was done from within the AutoGen framework, which represents our agent orchestration area.

Later, *load_profile*, *upsert_profile*, and *list_profiles* were implemented, which are functions brought from our file called *data_base_profiles*, which works as a backend to handle the data of each user account that logs into the developed website.

Streamlit was also installed locally so that the framework could later be imported for UI design, along with *st_audiorec*, a Streamlit component that enabled audio recording from the browser. Additionally, the *Whisper* library developed by OpenAI for transcribing audio to text was downloaded locally, along with *ffmpeg*, which was also installed on the test machines.

For translation, *GoogleTranslator* was imported from *deep_translator* to perform robust and reliable translation of the conversations. The *ElevenLabs* and *VoiceSettings* tools were used to convert the translated text to audio. Finally, *constraints* were incorporated to address the problem of language

304 restrictions. Certain barriers were established, such as requir- 358
305 ing the source and target languages to be different, or identi- 359
306 fying language pairs for which direct translation is not suffi- 360
307 ciently accurate. This approach aims to generate a set of bet- 361
308 ter translation solutions. Additionally, *type* annotations were 362
309 applied within the DFS function. These annotations help de- 363
310 fine the expected data structures, specifying, for instance, that 364
311 the path variable is a list of strings rather than leaving it as 365
312 None, thereby providing clearer guidance during the con- 366
313 struction and execution of the DFS function.

314 3.2 API Keys Block

315 Once the exposed code section was defined, we declared 370
316 the API keys corresponding to OpenRouter (all the ones used 371
317 in the experiments) and ElevenLabs. Afterwards, we build 372
318 the *model_client* through the model definition using the 373
319 aforementioned API key and entering the URL of the artifi- 374
320 cial instrument with different pre-established model parame- 375
321 ters. 376

322 3.3 Tools Block

323 Hereafter, we developed the nine tools that the agents use 379
324 to execute the entire system translation flow. It's worth noting 380
325 that we implemented most of these functions using try and 381
326 catch blocks to handle errors at different stages of the pro- 382
327 gram flow. First, we have *user_profile*, which uses the 383
328 *user_id* signature as a string, calling *load_profile(user_id)* 384
329 and returning a standardized dictionary *{ "profile": profile }*. 385
330 This makes the function act as an adapter between the profile 386
331 database and the rest of the application. Second, we devel- 387
332 oped the *CSP* tool, which uses the available languages and 388
333 voice options to compute valid translation configurations 389
334 based on the source language, target language, and voice gen- 390
335 der. The tool returns -1 if the source and target languages are 391
336 identical, -2 if the selected languages belong to restricted 392
337 pairs, or a set of possible solutions, from which one option is 393
338 randomly selected when multiple valid solutions exist. The 394
339 next component is *DFS*, which receives the language graph, 395
340 the source and target languages, and the set of visited nodes, 396
341 initially set to None. The *DFS* tool traverses the graph, re- 397
342 turning the path covered until it reaches the target language. 398
343 The *translate_along_path* function takes the user, transcript, 399
344 and the path returned by DFS, translating the transcript step 400
345 by step along the path until reaching the semifinal node. It 401
346 returns the translated transcript and the semifinal node as the 402
347 source, or the original transcript and source if no translation 403
348 is necessary. Finally, the *dfs_translate* tool serves as a sim- 404
349 plified interface for the agent, receiving the user ID and tran- 405
350 script. It loads the source and target languages from the user, 406
351 calls *DFS* and *translate_along_path* internally, and returns 407
352 either the translated transcript and intermediate source or the 408
353 original values, depending on whether the selected languages 409
354 fall within the defined language restrictions. Later on, it is 410
355 *auto_text_recognition*, which receives an *audio_file*, loading 411
356 the base version of the *Whisper* model (to handle a standard 412
357 data flow) and transcribing the received audio. It's notewor-

thy that *fp16* was set to false, a decision which was made be-
cause this floating point format can commonly cause errors
or lead to unusual behavior. Similarly, it's significant to
standout that *ffmpeg* was installed, as *Whisper* relies on this
format for audio-to-text conversion. The seventh function in-
troduced was *latency*, which uses the variables *text* and *pro-*
file. This by reading the maximum latency specified by the
user, and calculating the amount of text to be translated, and
therefore deciding whether to use a fast or high-quality trans-
lation mode. The *latency* tool, then returns a dictionary for
the agent involved in this function. The eight one was defined
as *machine_translation*, and its signature includes *user_id*. It
translates using the third-party tool *Google Translate*, load-
ing the user's information to determine the source and target
languages. Ultimately, the *text_to_speech* function was built,
which uses the third-party tool *ElevenLabs* to convert the
translated text into audio with different voice settings such as:
stability (0-1, emotional variation that allows the voice to
change between phrases and words), *similarity_boost* (0-1,
configuration that determines how much the audio attempts
to resemble the timbre of the base/trained voice), *style* (0-1,
it allows to select one of the predetermined voice styles or
genres from the audio model), *use_speaker_boost* (True or
False, if it is True then it allows for greater voice presence,
making it sound more solid, which is helpful in modest speak-
ers or noisy environments). While simultaneously writing the
file in parts using binary format through the use of a *for cycle*
getting those chunks (parts), that at the end form the mp3 au-
dio with all the bytes blocks.

387 3.4 Agents Block

388 Now we present the most prominent component of the sys-
tem: the definition and orchestration of agents. As detailed in
Section 2, each agent assumes a well-defined functional role
and communicates exclusively through JSON responses.
Specifically, the Profile Agent queries the user's profile and
returns *{ "profile": {...} }*, the ASR Agent processes the audio
and returns *{ "transcript": "...", "detected_language": "...",*
the Latency Agent estimates the quality as well as the target
latency and responds *{ "mode": "fast|quality",*
"time_left_ms": <int> }, the DFS Agent receives both the
transcript and the user profile, and returns a JSON structure
that mirrors the format produced by the ASR Agent. When
necessary, this structure is updated to reflect any detected
transitions, ensuring that downstream agents receive the most
coherent and context-aware information possible. The result-
ing output follows the same schema, *{ "transcript": "...", "de-*
tected_language": "...", "translated_text": "...", and the MT Agent translates the
content according to the policy specified by the Latency Man-
ager and produces *{ "translated_text": "...",* On the other
hand, the Summarizer Agent summarizes long texts in *{ "sum-*
mary": "...", and, finally, the TTS Agent synthesizes speech
from both the translation and the summary, returning:
TRANSLATION { "audio_path": "...", "text": "...", "sum-
mary": "...", Each of these agents explicitly uses the previ-
ously described local tools, thus keeping the LLM as an or-
chestrator and not as a logic executor.

The Agent management is handled through the definition of the *RoundRobinGroupChat* team, which defines the sequential order in which the agents run (first *profile_agent*, then *asr_agent*, then *latency_agent*, *dfs_agent*, *mt_agent*, *summarizer_agent*, and finally *tts_agent*). This is ensured by defining a maximum of seven interventions, one for each agent. The agent cycle is also defined to terminate with the word *TRANSLATION*, which is emitted by *tts_agent*, thus ending the translation tool's flow.

The execution of the agent group is performed via an asynchronous call `result = await Con-sole(team.run_stream(task=claim))`, which triggers the team's round robin chat with the claim as the input task. After that, the model client is closed with `await model_client.close()` and the consolidated result of the flow is returned, which is the JSON of the *tts_agent*.

3.5 Data Base Block

Leaving aside the main .py file for a moment, the tool used to manage all the data related to each user was a database implemented as a backend for the project in a file called *data_base_profiles.py*. This file imported the SQLite3 language using the command `import sqlite3`. First, the path to the database file was defined using `DB_PATH = "profiles.db"`, thus connecting the database to this variable that stores its path. Secondly, the *create_table* function was created, which generates the table if it doesn't already exist, using parameters that describe each user of the application: *user_id* (TEXT PRIMARY KEY), *source_language* (TEXT NOT NULL), *target_language* (TEXT NOT NULL), *voice_gen* (TEXT NOT NULL), *register* (TEXT NOT NULL), and *max_latency_ms* (INTEGER NOT NULL).

Thereupon, a second function called *upsert_profile* was created, which allows inserting or updating user data in the previously created table using SQLite functions such as *INSERT INTO* (to insert data) and *ON CONFLICT DO UPDATE* (to update data). In the third step, the *load_profile* function was developed, which reads a profile using a parameterized *SELECT* query, returning only one row via *fetchone()*. This fetchone is mapped to a dictionary with clear keys (*user_id*, *source_language*, *target_language*, *voice_gen*, *register*, *max_latency_ms*). This allows the main code to read and retrieve user data for later use in the requested translation. In the end, the *list_profiles* function was defined, which returns an alphabetically ordered list of ID's. This list is used to populate the "existing users" selectbox in the user interface, which will be explained later.

It is remarkable that in the SQLite mentioned layer, each call follows a clear cycle that ensures integrity and operational simplicity, this with the use of the following concrete functions that were used throughout all the code developed. First, *create_table()* guarantees that the schema exists before any read or write operation, preventing missing table failures and establishing the minimum data rules. Then, *get_conn()* opens the connection to the *profiles.db* file, the transactional anchor point upon which everything else is built. With that connection, *conn.cursor()* creates a cursor that prepares and

executes SQL statements, and *cur.execute(...)* implement it to navigate through the existence tables. After the modification operations, *conn.commit()* seals the transaction and makes the changes persistent and *conn.close()* releases resources and locks, preventing leaks and the classic "database is locked" message in reactive applications. The use of these set of functions from SQLite turns the database into a predictable, robust, and easy-to-maintain component.

3.6 Streamlit UI Block

Finally, and returning to the principal .py file, the user interface was created by defining it within the program's main function using the *Streamlit* tool, primarily based on its documentation. It's important to note that, since *Streamlit* is initially exported as *st*, all calls to functions within the interface development tool were made using the *st* prefix.

First, the title of the tab containing the translator was defined as "AI Live Translator" using the full width from the command `st.set_page_config("AI Live Translator", layout="wide")`. Then, keys were initialized in *st.session_state*, specifically those of *current_user* and *messages*, which allows that in each page rerun the current user and their message history are not lost, while also determining that if a user is not defined, they are marked as not existing along with their messages. Subsequently, the sidebar of the interface was created where the user who will use the web application is defined, this with the title "User Profile" with the line of code `st.sidebar.header("User Profile")`, with the creation of a bar that displays the users already stored or gives the option to create a new one, and with the appearance of a text area where the name of the user appears already loaded or allows to put the name of the user in case it is a new one.

In addition, the available languages for translation were defined using `languages_available = ["es", "en", "pt", "fr", "de", "it", "ja", "ru", "zh-CN"]`, allowing users to choose between Spanish, English, Portuguese, French, German, Italian, Japanese, Russian, and Mandarin. More bars were also added to the sidebar, allowing users to load or select, as needed, the user's speaking language, the target language, the grammar for formal or informal translation, the gender translation voice and the preferred latency, which ranges from 300ms to 3000ms. It's worth mentioning that in all these parameters, if any are not manually selected due to user oversight, a configuration was applied using if statements to apply predefined selections (Spanish as the speaking language, English as the translation language, formal grammar, male voice and 1500ms latency). This prevents errors caused by not obtaining all the necessary parameters for the program to function.

Later on, two constraints were also set calling the *csp* function with `languages_available` and `voice` parameters. Firstly, the function gives a code state "-1" when existing incompatibility between options (the voice does not support the target language). In the other case, a "-2" code state is given when the source and target languages are the same. Both constraints, are shown in the user interface through the *st.warning* function from Streamlit.

At the bottom of the sidebar, a button was placed to save the user's parameters or update those of an existing user using the command `st.sidebar.button("Save / Update")`. This involves a call to the database `upsert_profile(user_id_input, source, target, voice_gen, register, latency_ms)`, setting the defined user as the current user with `st.session_state["current_user"] = user_id_input`, and finally displaying a success message with `st.sidebar.success(f"Profile saved for {user_id_input}")`.

The web application's main page was initiated by defining its header as "AI Live Translator", accompanied by a decorative red bar using the command `st.header("AI Live Translator", divider="red")`. A gray subheader was also created to display our names as a signature for the creation of AI Live Translator using `st.subheader("gray by Carlos Lomelin and Sebastián Garmendia")`.

Below this, a welcome message was displayed with the code line `st.write(f"Welcome, **{st.session_state['current_user'] or 'guest'}**")`. Just after it, a for loop block iterates through `st.session_state["messages"]` so that in each message from the agent group, a chat-style container opens displaying their role, the translated text using `st.write(msg.get("content", ""))` and the audio, which is saved through the commands `ab = msg.get("audio_bytes", b"")` and `ap = msg.get("audio", "")` to first search for and play the audio in memory and if it has not been loaded, it does so through the file path and not from the bytes of the file.

Eventually, the audio recording area was created with the title `st.markdown("Recording")` and a call to Streamlit's default function `st.audio_rec()` was implemented to capture audio from the UI. This audio can be played, paused, or downloaded as a WAV file. Similarly, a button to translate the audio was implemented.

This button (after checking the not assessment of constraint -2) first ensures the existence of the audios folder with `os.makedirs("audios", exist_ok=True)` and then moves the audio path from the user input to the same folder with `input_path = os.path.join("audios", "input.wav")`. When this button is clicked, first the `user_id` is obtained using `user_id = st.session_state.get("current_user")` or `user_id_input` or `guest`. This ID is then used in the claim sent to the agents, clearly outlining the agent flow (profile → ASR → latency → DFS → MT → summarizer → TTS) and including a final guide requesting a response based on the TTS instructions, enabling the execution of the agents using the command `result = asyncio.run(run_ai_live_translator(claim))`.

Subsequently, the list of messages between agents was obtained using `msgs = getattr(result, "messages", [])` or `[]`, preventing errors in case no messages exist and thus working with an empty list. This allowed for traversing the agent messages from the end to the beginning, searching for the one generated by the TTS agent, which has the word "TRANSLATION" as its indicator. This was done by extracting the content with the `getattr` and `_to_text` functions and storing the TTS message in a variable called `payload`.

Once the message was stored, regex was also employed to capture the JSON following "TRANSLATION" so that the `translated_text` and `summary` could be used through getters. In addition, we employed a try-except logic to be able to handle a situation in which the model returns the message in single quotes (not the expected `json.loads()`) instead of double quotes with `ast.literal_eval()` for interpreting the received message as a style Python dictionary. Note that the `tts_path` was skipped because in all cases of our experiments the model employed generated whatever path it felt like, making it impossible to match the path addresses, so we decided on the audio path ourselves in Python (audios/traduc.mp3), and ignored the one generated by the model.

Later, two alternatives were offered, since there is always the possibility that the agent's flow might have a gap and therefore the translation/summary could not be extracted. In summary, the output of both the MT and Summarizer agents is read using the same logic as when searching for "TRANSLATION," but now for "translated text" and "summary". These code blocks come into play when the selected model unexpectedly fails to display the message containing the word "TRANSLATION", a situation we encountered several times due to the model's highly unpredictable behavior.

Afterwards, an alternative was developed for creating the audio by directly calling the `text_to_speech` tool in cases where `tts_path` was not generated, but `translated_text` was available, due to the problem aforementioned. Thus extracting the path of the locally generated MP3.

Additionally, a problem that arose twice during code development was mitigated: the audio would stop at 0:00 on the user interface. This was resolved by checking if the path existed, if the file was larger than 0, and then opening it in binary format and loading all the bytes into memory.

Furthermore, `content_text` was created to display the generated translation and a summary of it (in case of an error, it informs the user that the translation is unavailable).

Lastly, the `st.session_state["messages"].append` function saves a new message each time the translate button is pressed and then displays it using the for loop block that iterates through `st.session_state["messages"]`. Each of these messages consists of the "assistant" role, `content_text`, `tts_path`, and `audio_path`. Besides, with `st.rerun()` Streamlit refreshes the page and makes the newly translated message, along with its audio, appear in the chat immediately.

4 Experiments and Results

For this section, we selected a set of representative experiments that demonstrate the system's behavior under different practical situations. Each experiment focuses on a particular challenge or functionality of the proposed architecture, providing empirical evidence of how the system handles real-world translation tasks.

We carried out over 100 tests, but here we present the five core experiments that best illustrate the system's capabilities. A quantitative and qualitative analysis follows in the Discussion section.

4.1 Experiment 1: Language-Restriction Scenario

```

637 .....indirectCallResult(latency_agent) .....
638 .....indirectCallResult(latency_agent) .....
639 .....indirectCallResult(latency_agent) .....
640 .....indirectCallResult(latency_agent) .....
641 .....indirectCallResult(latency_agent) .....
642 .....indirectCallResult(latency_agent) .....
643 .....indirectCallResult(latency_agent) .....
644 .....indirectCallResult(latency_agent) .....
645 .....indirectCallResult(latency_agent) .....
646 .....indirectCallResult(latency_agent) .....
647 .....indirectCallResult(latency_agent) .....
648 .....indirectCallResult(latency_agent) .....

```

Figure 1. Experiment under language-restriction conditions.

The first experiment illustrates a language-restriction scenario where the use of the DFS agent becomes necessary. As shown in Figure 1, the ASR agent first generates the transcript from the audio input (“Hoy es 27 de noviembre”). Because a direct path from Spanish to Japanese is unavailable, the translation module transitions into an intermediate state. The DFS agent then computes the optimal route, which in this case is “es → en → ja,” producing an intermediate English output. This intermediate step enables the MT agent to operate correctly, ultimately delivering the final translation: “今日は 11 月 27 日です。”

The result preserves the meaning of the original sentence, confirming the effectiveness of the DFS-based routing mechanism.

4.2 Experiment 2: Constraint Blocking Translation

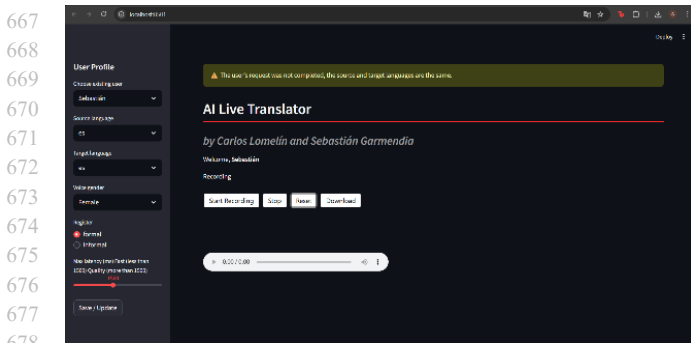


Figure 2. Constraints blocking the translation button.

In the second experiment, we evaluated the behavior of constraints in the user interface. Figure 2 shows a constraint that blocks the translation button when users select the same language as both source and target.

The system also displays a warning to inform the user of this issue. This mechanism prevents unnecessary requests and avoids spending computation resources on redundant translations, all through the implementation of a constraint satisfaction problem (CSP).

4.3 Experiment 3: Accuracy Constraint Warning

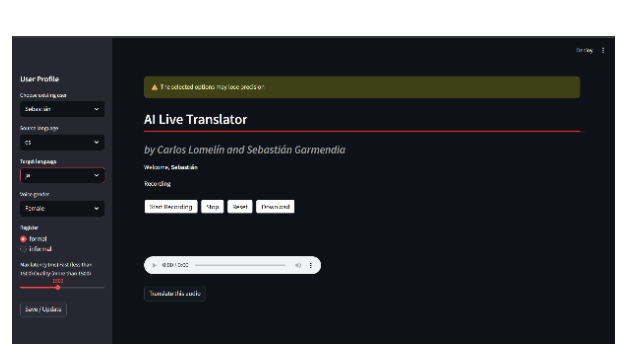


Figure 3. Constraints warning the accuracy.

A second constraint-related experiment is shown in Figure 3, where the interface warns the user about possible accuracy loss under certain combinations of source and target languages. This feature helps maintain user awareness of potential quality issues in advance, improving the reliability of the interaction.

4.4 Experiment 4: Translation with Summary Generation, specific voice gender and latency

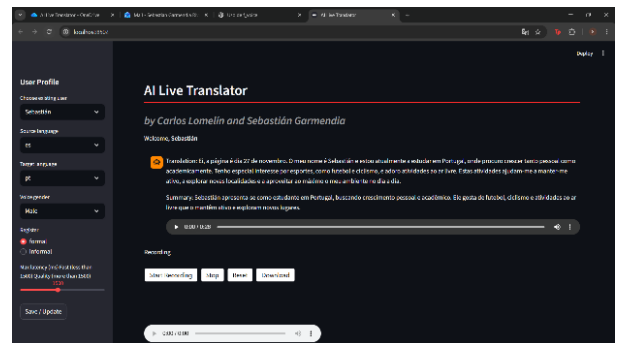


Figure 4. Translation with summary.

The fourth experiment demonstrates how the system manages long translations. In this case, a Spanish-to-Portuguese translation produces an extended output, and the summarizer agent automatically generates a condensed version of the content. This summary captures the essential meaning of the complete translation while significantly reducing textual length.

This summarization step is especially useful when processing long or highly descriptive passages, where a full translation may be excessive for the intended task. The summarizer maintains semantic coherence and contextual relevance, avoiding distortions or omissions. Consequently, combining translation and summarization enables faster multilingual information access, benefiting tasks such as document screening, content filtering, and rapid information retrieval.

It can also be seen in the displayed image that the voice gender was determined to be “male”, which allowed the voice generated by the third party tool (Whisper) to be successfully heard in the masculine gender.

748 Additionally, the latency time was set to 1500ms (as de-
749 fault) and the treated text was constituted of more than 80
750 characters, the MT Agent performed the translation in quality
751 mode, providing a more robust and consistent translation.

752 4.5 Experiment 5: Rate-Limit and Model Instabil- 753 ity Issues

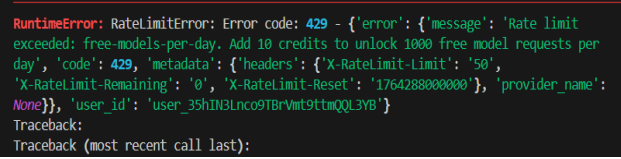
```
754 
755 RuntimeError: RateLimitError: Error code: 429 - {'error': {'message': 'Rate limit
756 exceeded: free-models-per-day. Add 10 credits to unlock 1000 free model requests per
757 day', 'code': 429, 'metadata': {'headers': {'X-RateLimit-Limit': '50',
758 'X-RateLimit-Remaining': '0', 'X-RateLimit-Reset': '1764288000000'}, 'provider_name':
759 None}}, 'user_id': 'user_35hIN3Lnc09TBvmt9ttwQQL3YB'}
760 Traceback:
761 Traceback (most recent call last):
```

Figure 5. Rate limit error.

762 The last experiment (Figure 5) exposes system limitations
763 related to the use of a free model via the OpenRouter API.
764 The system displays a rate limit error, which prevents further
765 requests even when unused credits remain. This constraint
766 significantly affects projects relying on free models, as global
767 usage limits can halt functionality unpredictably.

768 Another observed issue concerns model instability. Across
769 more than 100 tests, the agents produced inconsistent outputs
770 occasionally failing to follow instructions such as quotation
771 formats, capitalization rules, or JSON structure (especially in
772 the TRANSLATION final message). These inconsistencies
773 particularly affected audio retrieval tasks, where unpredicta-
774 ble file names or formatting forced additional validation
775 steps. We expect these issues to diminish when using paid
776 models, which offer higher stability, no rate limits, and im-
777 proved adherence to specified formats.

778 5 Discussion

779 After performing over 100 experiments across multiple
780 languages, contexts, and constraint configurations, we con-
781 firm that the system consistently fulfills its primary objective:
782 receiving audio as input and returning a translated audio out-
783 put. The experiments presented demonstrate that the architec-
784 ture operates correctly across different translation paths, in-
785 terface conditions, and content lengths.

786 To evaluate translation quality, we applied both qualitative
787 and quantitative metrics.

788 From a qualitative perspective, we examined three main
789 aspects. First, we assessed fidelity, observing that the trans-
790 lations consistently preserved the meaning of the original in-
791 put throughout all the experiments. Second, we evaluated nat-
792 uralness, a critical factor given that non-native speakers often
793 produce grammatically correct but unnatural expressions. In
794 contrast, our system generated outputs that aligned well with
795 native language patterns, demonstrating fluent and contextu-
796 ally appropriate formulations. In third instance, we analyzed
797 context preservation, confirming that the system maintained
798 coherent phrasing even when producing summaries of long
799 inputs, avoiding distortions or loss of essential information.

800 From a quantitative standpoint, we employed back-trans-
801 lation as the primary evaluation method. We compared the

802 back-translated text with the original input, counted the dis-
803 crepancies, and calculated their proportion relative to the total
804 number of words. The resulting error rate remained consist-
805 ently low, confirming the accuracy of most translations. We
806 also computed the number of completely error-free transla-
807 tions versus those that contained any degree of inconsistency.
808 Interestingly, this second measure produced an even lower
809 error rate because several discrepancies often originated from
810 a single earlier error in the translation chain.

811 To illustrate the application of the evaluation metrics, we
812 consider the following case: The original audio contained the
813 message: "Hola, hoy es 27 de noviembre. Mi nombre es Se-
814 bastián y actualmente estoy estudiando en Portugal, donde
815 busco crecer tanto personal como académicamente. Me in-
816 teresan especialmente los deportes, como el fútbol y el ci-
817 clismo, y disfruto mucho de las actividades al aire libre. Estas
818 actividades me ayudan a mantenerme activo, explorar nuevos
819 lugares y aprovechar al máximo el entorno en mi día a día."

820 The system produced the translation: "Ei, a página é dia 27
821 de novembro. O meu nome é Sebastián e estou a estudar em
822 Portugal, onde procuro crescer tanto pessoal como academi-
823 camente. Tenho especial interesse por esportes, como futebol
824 e ciclismo, e adoro atividades ao ar livre. Estas atividades
825 ajuma-me a manter-me ativo, a explorar novas localidades e
826 a aproveitar ao máximo o meu ambiente no dia a dia."

827 In terms of fidelity, we observe that the semantic essence
828 is adequately preserved: the core concepts, communicative
829 intent, and overall structure of the message remain intact, al-
830 lowing us to consider the translation highly faithful to the
831 original content.

832 Regarding naturalness, the system employs constructions
833 characteristic of European Portuguese, such as "estou a estu-
834 dar", and selects contextually appropriate vocabulary. Alt-
835 hough some choices are less natural, for example: the use of
836 "esportes" instead of "desportos", these represent only a small
837 proportion of the total, resulting in an overall satisfactory
838 level of naturalness.

839 For the quantitative evaluation, we applied back-transla-
840 tion. The resulting text was: "Hola, la página es del 27 de no-
841 viembre. Me llamo Sebastián y estoy estudiando en Portugal,
842 donde busco crecer tanto personal como académicamente.
843 Me interesan especialmente los deportes, como el fútbol y el
844 ciclismo, y me encantan las actividades al aire libre. Estas ac-
845 tividades me ayudan a mantenerme activo, a explorar nuevos
846 lugares y a disfrutar al máximo de mi entorno en el día a día."

847 The main error detected corresponds to the phrase "la
848 página es del", which is not a natural translation back into
849 Spanish. This discrepancy accounts for four words differing
850 from the reference, resulting in a quantitative error rate of
851 4/66 for this sample. Under the second quantitative metric,
852 which counts only whether a translation contains an error, re-
853 gardless of its length this case is counted as a single error.
854 Across the broader set of experiments, most translations were
855 error-free under this metric, confirming consistently accurate
856 performance.

858 Taken together, both qualitative and quantitative findings
859 indicate that the system effectively addresses the core prob-
860 lem of real-time multilingual audio translation.

861 Despite these strengths, the system is not free of limita-
862 tions. As stated in the last section, the most significant con-
863 straints arise from the use of free models, which introduce
864 rate-limit restrictions, inconsistent adherence to instructions,
865 and occasional instability in the JSON structure returned by
866 the agents. Additionally, as with any microphone-based ap-
867 plication, overall translation quality depends heavily on audio
868 clarity, higher quality microphones naturally produce better
869 results. Nevertheless, because the development process be-
870 gan early, we were able to conduct a sufficient number of ex-
871 periments to perform a meaningful evaluation despite these
872 constraints.

873 6 Conclusions

874 In relation to the project carried out, we are capable of ensur-
875 ing that we provide a tool as a comprehensive technological
876 solution to democratize multilingual communication in real
877 time.

878 The AI Live Translator is constituted by a multi-agent sys-
879 tem, that coordinates speech detection, automatically trans-
880 late, and control terminology, with strict latency management
881 and high translation quality. This was achieved with a mini-
882 malist and practical chat-based design that allows for fluid
883 communication between users.

884 Furthermore, the qualitative and quantitative metrics of the
885 evaluation plan put forward, and the several evaluations per-
886 formed ensure that the quality that is sought provides the us-
887 ers with reliance on the instrument they employ.

888 A key opportunity to enhance functionality in this project,
889 would be the implementation of domain-specific translation
890 engines or tools from sectors such as business, engineering,
891 and medicine, among others. This would allow for the provi-
892 sion of translations that handle highly technical vocabulary
893 and concepts in these types of contexts. This, in turn, would
894 enable better performance in much more specific and tech-
895 nical environments. Moreover, an improvement could proba-
896 bly also be implemented in terms of the response speed of the
897 translations, since in our case it is considerably slow due to
898 the use of the "alibaba/tongyi-deeppresearch-30b-a3b:free"
899 model which is a free version and depends directly on
900 whether it is saturated or not as a consequence to the use be-
901 ing given to it at a specific time around the world.

902 It is noteworthy that the above are just suggestions among
903 many possibilities, however, the advancement of AI-powered
904 translation is happening at a rapid pace, making the potential
905 for improvement increasingly subtle and likely only recog-
906 nizable by experts in languages and terminology.

907 Separately and beyond functionality, another area we iden-
908 tified with room for improvement is the design, but not the
909 functionality of the developed UI, since it provides a smooth
910 performance in terms of translations but could be more re-
911 fined in terms of the visuals, clearer layouts or distribution of
912 the web application canvas.

913 In summary, this project provides an aspirational tool to
914 reduce and assist in eliminating the language barrier in real-
915 life interactions, especially in academic and professional con-
916 texts. The system emphasizes usability and reliability while
917 leaving room for continued refinement of interface layout and
918 specialized glossaries. Ultimately, it can be stated that AI
919 Live Translator is a utensil which represents a solid founda-
920 tion for language problem-solving, thus possessing the great
921 possibility of evolving through targeted model updates and
922 feedback loops, helping users communicate with greater con-
923 fidence and precision.

924 References

925 Amazon Web Services. n.d. "Amazon Polly (Text-to-
926 Speech)." AWS AI Services. Available at:
927 <https://aws.amazon.com/es/polly/> (accessed October 13,
928 2025).

929 Biswas, D. 2024. Constraints Enabled Autonomous Agent
930 Marketplace: Discovery and Matchmaking. In *Proceed-
931 ings of the 16th International Conference on Agents and
932 Artificial Intelligence (ICAART 2024)*, 396–403. SciTe-
933 Press.

934 ElevenLabs. 2025. Free AI Voice Generator & Voice Agents
935 Platform. elevenlabs.io. Available at: [https://eleven-
936 labs.io/](https://eleven-labs.io/) (accessed October 29, 2025).

937 ElevenLabs. 2025. text_to_speech_file.py. In elevenlabs-ex-
938 amples (GitHub repository). Available at:
939 [https://github.com/elevenlabs/elevenlabs-exam-
942 ples/blob/main/examples/text-to-speech/py-
943 thon/text_to_speech_file.py](https://github.com/elevenlabs/elevenlabs-exam-
940 ples/blob/main/examples/text-to-speech/py-
941 thon/text_to_speech_file.py) (accessed November 1,
944 2025).

945 Europa Press. 2019. "Los idiomas, en cifras: ¿cuántas lenguas
946 hay en el mundo?" [Languages in numbers: How many
947 languages are there in the world?]. Available at:
948 [https://www.europapress.es/sociedad/noticia-idiomas-
951 cifras-cuantas-lenguas-hay-mundo-
952 20190221115202.html](https://www.europapress.es/sociedad/noticia-idiomas-
949 cifras-cuantas-lenguas-hay-mundo-
950 20190221115202.html) (accessed October 11, 2025).

953 Ghosh, Bijit. n.d. "Agentic Design Patterns." Medium. Avail-
954 able at: [https://medium.com/@bijit211987/agentic-de-
956 sign-patterns-cbd0aac2962f](https://medium.com/@bijit211987/agentic-de-
955 sign-patterns-cbd0aac2962f) (accessed October 11, 2025).

957 Google Cloud. n.d. "Choose a design pattern for your agentic
958 AI system." Cloud Architecture Center. Available at:
959 [https://cloud.google.com/architecture/choose-design-pat-
961 tern-agentic-ai-system](https://cloud.google.com/architecture/choose-design-pat-
960 tern-agentic-ai-system) (accessed October 11, 2025).

962 Google Cloud. n.d. "Cloud Text-to-Speech." Cloud Text-to-
963 Speech Documentation. Available at:
964 <https://cloud.google.com/text-to-speech> (accessed Octo-
965 ber 13, 2025).

966 Microsoft. n.d. "Azure AI Speech." Azure – AI Services.
967 Available at: [https://azure.microsoft.com/en-us/prod-
969 ucts/ai-services/ai-speech](https://azure.microsoft.com/en-us/prod-
968 ucts/ai-services/ai-speech) (accessed October 12, 2025).

970 Nisa, U.; Shirazi, M.; Saip, M. A.; and Pozi, M. S. M. 2025.
971 Agentic AI: The age of reasoning—A review. *Journal of*

965 Automation and Intelligence.
966 <https://doi.org/10.1016/j.jai.2025.08.003>

967 Pertusa, Inmaculada (ed.). n.d. “1. La comunicación y sus
968 componentes.” In *Composición y gramática en español*
969 (Pressbooks, Western Kentucky University). Available at:
970 [https://saalck.pressbooks.pub/spanish-composition-and-](https://saalck.pressbooks.pub/spanish-composition-and-gram-mar-wku/chapter/a-1-la-comunicacion-y-sus-com-ponentes/)
971 [gram-mar-wku/chapter/a-1-la-comunicacion-y-sus-com-](https://saalck.pressbooks.pub/spanish-composition-and-gram-mar-wku/chapter/a-1-la-comunicacion-y-sus-com-ponentes/)
972 [ponentes/](https://saalck.pressbooks.pub/spanish-composition-and-gram-mar-wku/chapter/a-1-la-comunicacion-y-sus-com-ponentes/) (accessed October 12, 2025).

973 Raheem, T., and Hossain, G. 2025. Agentic AI Systems: Op-
974 portunities, Challenges, and Trustworthiness. In *Proceed-*
975 *ings of the 2025 IEEE International Conference on Elec-*
976 *tro Information Technology (eIT)*, 618–624. IEEE.

977 Schmid, Philipp. n.d. “Agentic Patterns.” philschmid.de.
978 Available at: <https://www.philschmid.de/agentic-pattern>
979 (accessed October 12, 2025).

980 Singh, Sumit. 2024. “6 Best Automatic Speech Recognition
981 (ASR) Tools.” Labellerr AI Blog, December 4, 2024.
982 Available at: [https://www.labellerr.com/blog/best-auto-](https://www.labellerr.com/blog/best-automatic-speech-recognition-tools-asr/)
983 [matic-speech-recognition-tools-asr/](https://www.labellerr.com/blog/best-automatic-speech-recognition-tools-asr/) (accessed October
984 11, 2025).

985 Streamlit. 2025. *Streamlit documentation*. docs.streamlit.io.
986 Available at: <https://docs.streamlit.io/> (accessed Novem-
987 ber 7, 2025).

988 Universitat Oberta de Catalunya (UOC) – Arts i Humanitats.
989 2024. Traducción e interpretación en la era de la tecnolo-
990 gía y la IA: retos, competencias y nuevos perfiles profes-
991 ionales. Blog post. Available at:
992 [https://blogs.uoc.edu/humanitats/es/traduccion-interpre-](https://blogs.uoc.edu/humanitats/es/traduccion-interpretacion-era-tecnologia-ia/)
993 [tacion-era-tecnologia-ia/](https://blogs.uoc.edu/humanitats/es/traduccion-interpretacion-era-tecnologia-ia/) (accessed October 27, 2025).